



ADVANCED DEEP LEARNING

MODULE 7: GENERATIVE ADVERSARIAL NETWORK

BITS Pilani

SEETHA PARAMESWARAN
WILPD - CSIS



The instructor is gratefully acknowledging the authors who made their course materials freely available online.

This deck is prepared by Seetha Parameswaran.

TABLE OF CONTENTS



1 Motivation & Big Picture

2 GAN Foundations

3 GAN Variants

4 Case Study: AV Prototype

5 Wrap-Up



WHY GANs? THE SAMPLING PROBLEM

- **Goal:** sample from complex, high-dimensional $p_{\text{data}}(x)$; no direct way to do this
- **Solution:** sample from simple $p(z)$ (Gaussian/uniform); learn transformation to p_{data}
- **Likelihood-based models so far:**
 - ▶ **AR:** exact $\log p$ — but slow sampling, $O(D)$ sequential passes
 - ▶ **Flows:** exact $\log p$ — but restricted invertible architecture
 - ▶ **VAE:** fast sampling — but optimizes ELBO, blurry samples
- **Common aspect:** all require explicit density estimation
- **GAN key insight:** learn the transformation *without* explicit density — implicit model
 - ▶ No $\log p_{\theta}(x)$ ever computed
 - ▶ Quality driven by adversarial feedback from a discriminator

EXPLICIT VS IMPLICIT DENSITY MODELS

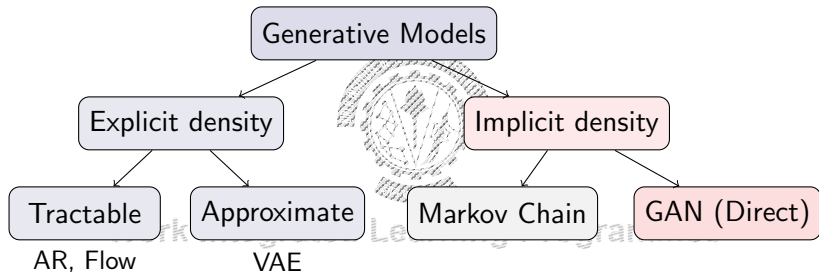
Explicit density

- Specify form of $p_{\theta}(x)$ directly
- Have tractable (or approximate) likelihoods
- Use an explicit density function
- Assume prior distribution over data
- **Examples:** AR, Flow, VAE

Implicit density

- Specify a stochastic *process* to simulate data
- No explicit density function
- $p_{\theta}(x)$ defined only through samples $x = G(z)$
- **Examples:** GAN

EXPLICIT VS IMPLICIT DENSITY MODELS II



THE ADVERSARIAL GAME



- **Generator NN** G_θ :

- ▶ Input: random noise $z \sim p(z)$
- ▶ Output: fake sample $\hat{x} = G_\theta(z)$
- ▶ **Never sees real data**
- ▶ Goal: fool D into predicting $D(\hat{x}) \rightarrow 1$

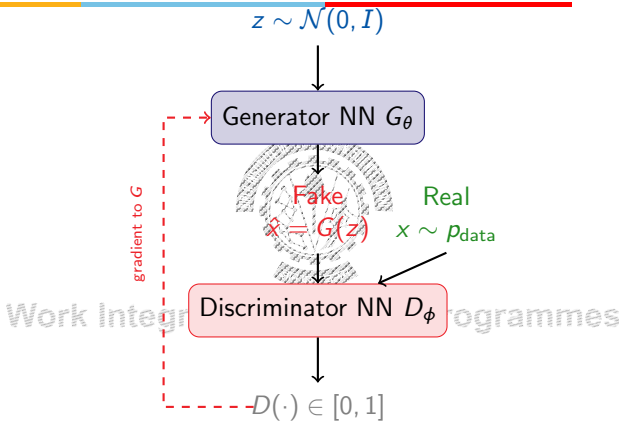
- **Discriminator NN** D_ϕ :

- ▶ Input: either real x or fake $\hat{x}$
- ▶ Output: probability of being real $\in [0, 1]$
- ▶ Goal: Predict $D(x) \rightarrow 1$ (real), $D(\hat{x}) \rightarrow 0$ (fake)

- **Minimax game**: both trained simultaneously, competing against each other

- After training: **discard** D , deploy G alone

THE ADVERSARIAL GAME



- G wants $D(\hat{x}) \rightarrow 1$ (fool discriminator)
- D wants $D(x) \rightarrow 1, D(\hat{x}) \rightarrow 0$ (correct classification)
- After training: discard D , use G for generation

GENERATOR: ROLE, ARCHITECTURE, OBJECTIVE

- **Role:** produce fake samples that look real
- **Input:** random noise vector $z \sim p(z)$
 - ▶ $p(z)$ is fixed — Gaussian or uniform
 - ▶ z is the only source of randomness
- **Architecture:** deep neural network
 - ▶ Learns mapping $G_\theta : z \mapsto \hat{x}$
 - ▶ Similar to VAE decoder — no encoder
 - ▶ For images: transposed convolutions (upsample)
- **What it captures:** $P(X | z)$ — features given noise
- **Objective:** fool D — maximize $D(G(z))$
- **Never sees real data** — learns only from D 's feedback

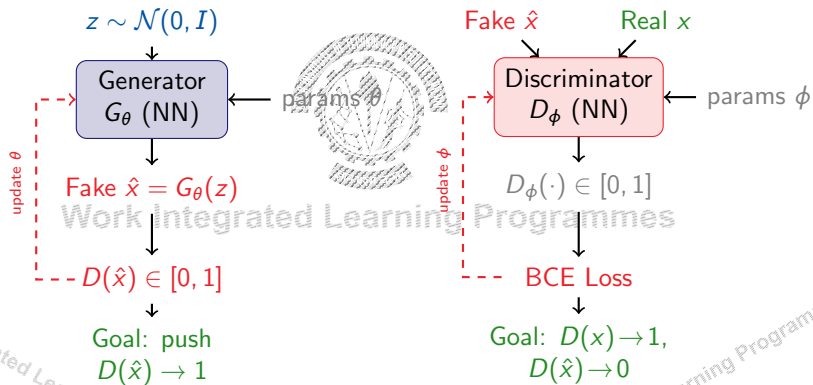
$$\max_{\theta} \mathbb{E}_{z \sim p(z)} [\log D_{\phi}(G_{\theta}(z))]$$

DISCRIMINATOR: ROLE, ARCHITECTURE, OBJECTIVE

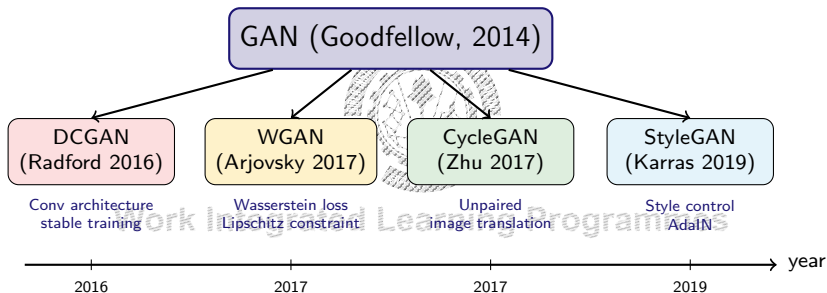
- **Role:** binary classifier — distinguish real from fake
- **Input:** either real $x \sim p_{\text{data}}$ or fake $\hat{x} = G(z)$
 - ▶ Labels: real = 1, fake = 0
- **Architecture:** deep neural network
 - ▶ Output: scalar $D_\phi(x) \in [0, 1]$ via sigmoid
 - ▶ For images: strided convolutions (downsample)
- **What it captures:** $P(Y | X)$ — class given features
- **Objective:** maximize binary cross-entropy
- Its output is **feedback signal** to train G
- **Discarded after training** — only G is deployed

$$\max_{\phi} \mathbb{E}_x[\log D_\phi(x)] + \mathbb{E}_z[\log(1 - D_\phi(G(z)))]$$

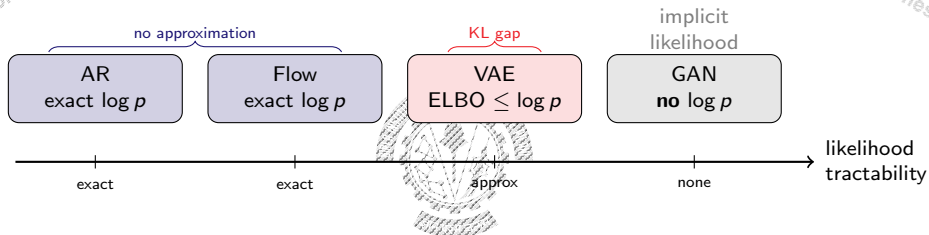
GENERATOR AND DISCRIMINATOR



GAN FAMILY TREE



ELBO THREAD: WHERE GAN SITS



- **GAN**: no ELBO, no explicit $\log p_{\theta}(x)$ — implicit distribution via $G(z)$
- Minimizes **Jensen-Shannon divergence** between p_{data} and p_g implicitly
- **Pros**: beautiful, state-of-the-art samples; fast inference
- **Cons**: unstable training; cannot evaluate $p(x)$, $p(z|x)$

TABLE OF CONTENTS



1 Motivation & Big Picture

2 GAN Foundations

3 GAN Variants

4 Case Study: AV Prototype

5 Wrap-Up



COMBINED TRAINING OBJECTIVE



- G and D have **opposing objectives** — combine into one minimax game:

$$\min_G \max_D V(D, G) = \underbrace{\mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)]}_{D \text{ on real data}} + \underbrace{\mathbb{E}_{z \sim p(z)} [\log(1 - D(G(z)))]}_{D \text{ on fake data}}$$

- **Discriminator** $\max_D$: push $D(x) \rightarrow 1$ and $D(G(z)) \rightarrow 0$
- **Generator** $\min_G$: push $D(G(z)) \rightarrow 1$, i.e. minimize $\log(1 - D(G(z)))$
- Equivalent to **binary cross-entropy** on real/fake labels:
 - ▶ Real data x has label $y = 1 \Rightarrow \log D(x)$
 - ▶ Fake data $G(z)$ has label $y = 0 \Rightarrow \log(1 - D(G(z)))$
- **Key: both networks trained simultaneously — D tries to maximize V , G tries to minimize V**

WHY DOES GAN WORK? THE KEY QUESTION

- We have defined the minimax objective — but **what is G actually minimizing?**
- To answer this, we ask: **what is the best possible D for a given G ?**
- Think of D as a **posterior probability estimator**:
 - ▶ $D(x)$ estimates: “given I see x , how likely is it real?”
 - ▶ If $p_{\text{data}}(x)$ is high and $p_g(x)$ is low $\Rightarrow D(x)$ close to 1
 - ▶ If $p_g(x)$ is high and $p_{\text{data}}(x)$ is low $\Rightarrow D(x)$ close to 0
 - ▶ If both are equal $\Rightarrow D(x) = \frac{1}{2}$ — **cannot distinguish real from fake**
- **Convergence intuition**:
 - ▶ At the start: $p_g \neq p_{\text{data}}$ — D can easily separate real from fake
 - ▶ As G improves: $p_g \rightarrow p_{\text{data}}$ — D struggles more and more
 - ▶ At convergence: $p_g = p_{\text{data}}$ — $D(x) = \frac{1}{2}$ everywhere
 - ▶ **This is the Nash equilibrium of the game**

OPTIMAL DISCRIMINATOR D^* I



- Fix G ; find $D^*(x)$ that maximizes $V(D, G)$
- Rewrite objective as integral — maximize pointwise over x :

$$V(G, D) = \int_x \left[p_{\text{data}}(x) \log D(x) + p_g(x) \log(1 - D(x)) \right] dx$$

- For any fixed x , maximize $f(y) = a \log y + b \log(1 - y)$:

$$\frac{\partial f}{\partial y} = \frac{a}{y} - \frac{b}{1-y} = 0 \implies y^* = \frac{a}{a+b}$$

- Substituting $a = p_{\text{data}}(x)$, $b = p_g(x)$:

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}$$

OPTIMAL DISCRIMINATOR D^* II

innovate

achieve

lead

- Confirms intuition:

- ▶ $p_{\text{data}} \gg p_g \Rightarrow D^*(x) \rightarrow 1$
- ▶ $p_g \gg p_{\text{data}} \Rightarrow D^*(x) \rightarrow 0$
- ▶ $p_g = p_{\text{data}} \Rightarrow D^*(x) = \frac{1}{2}$

Nash equilibrium reached

WHAT IS G MINIMIZING? JS DIVERGENCE

• Jensen-Shannon Divergence — intuition first:

- ▶ KL divergence $D_{\text{KL}}(p||q)$ is **asymmetric** —
 $D_{\text{KL}}(p||q) \neq D_{\text{KL}}(q||p)$
- ▶ JSD is the **symmetric, smoothed version**: use mixture
 $m = \frac{p+q}{2}$ as reference

$$D_{\text{JS}}(p||q) = \frac{1}{2} D_{\text{KL}}(p||m) + \frac{1}{2} D_{\text{KL}}(q||m), \quad m = \frac{p+q}{2}$$

- ▶ Always bounded: $D_{\text{JS}} \in [0, \log 2]$

• Substitute D^* into $V(G, D^*)$:

$$V(G, D^*) = -\log 4 + 2 D_{\text{JS}}(p_{\text{data}} || p_g)$$

WHAT IS G MINIMIZING? JS DIVERGENCE II

- Global minimum = $-\log 4$ iff $p_g = p_{\text{data}}$ $\Rightarrow$ **GAN recovers training distribution**
- **Compare with MLE:** MLE minimizes $D_{\text{KL}}(p_{\text{data}} \| p_g)$ — asymmetric, causes blurry samples
GAN minimizes symmetric D_{JS} — **sharper, more realistic samples**
- **Practical problem:** when p_g and p_{data} have non-overlapping supports, $D_{\text{JS}} = \log 2$ (constant) $\Rightarrow$ **zero gradient to G** — motivates WGAN

ALTERNATING TRAINING



- Cannot optimize $\min_G \max_D V$ jointly — alternate between updates

- **Step 1 — Update D (k steps, gradient ascent):**

$$\max_{\phi} \left[\mathbb{E}_x[\log D_{\phi}(x)] + \mathbb{E}_z[\log(1 - D_{\phi}(G_{\theta}(z)))] \right] \quad (\text{freeze } G)$$

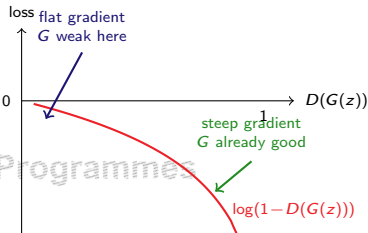
- **Step 2 — Update G (1 step, gradient descent):**

$$\min_{\theta} \mathbb{E}_z[\log(1 - D_{\phi}(G_{\theta}(z)))] \quad (\text{freeze } D)$$

- Typically $k = 1$ in practice — equal update frequency
- **Problem with Step 2:** gradient of $\log(1 - D(G(z)))$ is flat when G is weak — addressed next

THE SATURATING GENERATOR PROBLEM

- Generator minimizes $\log(1 - D(G(z)))$
- **Problem:** when G is weak early in training, $D(G(z)) \approx 0$
 - ▶ This is where G needs to *learn most*
 - ▶ But gradient of $\log(1 - D(G(z)))$ is **flat here**
- Gradient signal **dominated** by region where $D(G(z)) \approx 1$ — where G is *already good*
- Result: **slow, unstable learning** early in training



GAN TRAINING ALGORITHM



- Minibatch SGD, alternating updates (Goodfellow et al., 2014):

Algorithm 1: Minibatch SGD training of GAN

Input: Mini-batch size m , discriminator steps k , iterations T

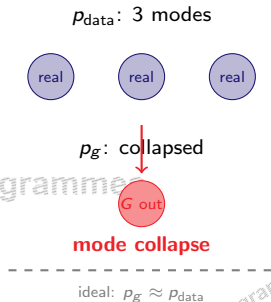
```
1 for  $t = 1$  to  $T$  do
2   for  $k$  steps do
3     Sample  $\{z^{(i)}\}_{i=1}^m \sim p(z)$ , Sample  $\{x^{(i)}\}_{i=1}^m \sim p_{\text{data}}(x)$ 
4     Update  $D$  by ascending:
        
$$\nabla_{\phi} \frac{1}{m} \sum_{i=1}^m \left[ \log D_{\phi}(x^{(i)}) + \log \left( 1 - D_{\phi}(G_{\theta}(z^{(i)})) \right) \right]$$

5     Sample  $\{z^{(i)}\}_{i=1}^m \sim p(z)$ 
6     Update  $G$  by ascending (non-saturating):  $\nabla_{\theta} \frac{1}{m} \sum_{i=1}^m \log D_{\phi}(G_{\theta}(z^{(i)}))$ 
```

- $k = 1$ most common; $k > 1$ gives D more updates per G step.

TRAINING INSTABILITY & FAILURE MODES

- **Mode collapse:** G maps all z to one or few modes
 - ▶ G exploits D 's blind spot
 - ▶ Low variance across z samples
- **Vanishing gradient:** if D too strong early, $\log(1 - D(G(z))) \approx 0$ — no signal to G
- **Oscillation:** G and D cycle without converging — no Nash equilibrium reached
- **Root cause:** JS divergence is constant when p_g and p_{data} have non-overlapping supports $\Rightarrow$ motivates WGAN



EVALUATING GANs: INCEPTION SCORE

- **Problem:** no explicit $\log p_{\theta}(x)$ — cannot use test-set likelihood to evaluate GANs
- **Idea:** good generators produce samples that are *semantically diverse* and *recognizable*
- Use pretrained **Inception-v3** network: $p(y|x)$, $y \in \{1000 \text{ ImageNet classes}\}$
- **Two criteria:**
 - ▶ Each image should be clearly recognizable $\Rightarrow p(y|x)$ should have **low entropy**
 - ▶ Generator should cover many classes $\Rightarrow$ marginal $p(y)$ should have **high entropy**

EVALUATING GANs: INCEPTION SCORE II

- Inception Score:

$$IS = \exp\left(\mathbb{E}_{x \sim p_g} \left[D_{KL}(p(y|x) \| p(y)) \right]\right) = \exp(H(y) - H(y|x))$$

- Higher IS = better
- **Limitation:** does not compare to real data distribution — a model generating one image per class can still score perfectly

EVALUATING GANs: FRÉCHET INCEPTION DISTANCE I

- Proposed to fix IS limitation — explicitly compares real and generated distributions (Heusel et al., 2017)
- Embed both real and fake images into Inception-v3 pool3 feature space (2048-d activations)
- Fit a Gaussian to each:
 - ▶ Real: mean m , covariance C
 - ▶ Generated: mean m_w , covariance C_w

- **FID formula:**

$$\text{FID} = \|m - m_w\|_2^2 + \text{Trace}(C + C_w - 2(CC_w)^{1/2})$$

- Lower FID = better — zero only when distributions are identical

EVALUATING GANs: FRÉCHET INCEPTION DISTANCE II

- Captures both **quality** (sharpness) and **diversity** (coverage of modes)
- **Limitation**: assumes Gaussian features; sensitive to number of samples used

NUMERICAL PROBLEM I



Let $p_{\text{data}}(x) = \mathcal{N}(0, 1)$ and $p_g(x) = \mathcal{U}(-2, 2)$. At $x = 0$: $p_{\text{data}}(0) \approx 0.399$, $p_g(0) = 0.25$.

- (a) Compute $D^*(0)$.
- (b) Compute $m(0) = \frac{1}{2}(p_{\text{data}}(0) + p_g(0))$.
- (c) Compute the KL contribution at $x = 0$: $p_{\text{data}}(0) \log \frac{p_{\text{data}}(0)}{m(0)}$.
- (d) What does $D^*(0) > 0.5$ imply?

NUMERICAL PROBLEM II



Solution:

$$(a) D^*(0) = \frac{0.399}{0.399 + 0.25} = \frac{0.399}{0.649} \approx 0.615$$

$$(b) m(0) = \frac{1}{2}(0.399 + 0.25) = 0.3245$$

$$(c) 0.399 \times \log \frac{0.399}{0.3245} = 0.399 \times \log(1.229) \approx 0.399 \times 0.206 \approx 0.082 \text{ nats}$$

(d) $D^*(0) > 0.5$ — discriminator correctly identifies $x = 0$ as more likely real than fake. Distributions have not yet converged: $p_g \neq$

p_{data}

DESIGN QUESTION I



Scenario: You are training a GAN to synthesize rare fault events (wheel slip, sudden brake failure) from AV telemetry logs. After 500 epochs the generator produces only one fault type regardless of the noise input z .

- (a) What failure mode is this? How would you detect it quantitatively?
- (b) Propose two architectural or training-level fixes.
- (c) Would WGAN help here? Why?

DESIGN QUESTION II



Discussion:

(a) **Mode collapse.** Detect by:

- (i) low variance of $G(z)$ across many z samples;
- (ii) high FID score — collapsed output diverges from real fault distribution.

(b) Fix 1: **Minibatch discrimination** — D receives batch statistics, penalizing identical outputs across the batch.

Fix 2: **Feature matching** — train G to match intermediate D feature statistics of real data, not just fool the final output.

(c) Yes — WGAN replaces JS divergence with Wasserstein distance, which provides meaningful gradients even when p_g and p_{data} have non-overlapping supports, directly addressing the vanishing gradient root cause of mode collapse.

TABLE OF CONTENTS



1 Motivation & Big Picture

2 GAN Foundations

3 GAN Variants

4 Case Study: AV Prototype

5 Wrap-Up



WASSERSTEIN GAN (WGAN) I



- **Problem:** JS divergence is constant ($= \log 2$) when p_g and p_{data} have non-overlapping supports $\Rightarrow$ zero gradient to G , unstable training
- **Key idea:** replace JS divergence with Earth Mover's (Wasserstein-1) distance
 - ▶ Intuition: minimum cost of transporting mass from p_g to p_{data}
 - ▶ Continuous and differentiable almost everywhere
 - ▶ Provides meaningful gradient even when supports don't overlap
- **Architecture change:** replace discriminator with a critic f_w
 - ▶ Critic outputs a scalar score, not a probability
 - ▶ Must be 1-Lipschitz: $|f(x_1) - f(x_2)| \leq |x_1 - x_2|$
 - ▶ Enforced by weight clipping to $[-c, c]$

WASSERSTEIN GAN (WGAN) II



- **WGAN objective:**

$$\min_G \max_{f \in 1\text{-Lip}} \mathbb{E}_{x \sim p_{\text{data}}} [f(x)] - \mathbb{E}_{z \sim p(z)} [f(G(z))]$$

- **Achieves:** stable training, meaningful loss curve, reduced mode collapse

WGAN-GP: GRADIENT PENALTY I

innovate

achieve

lead

- **Problem with weight clipping:**

- ▶ Underuses network capacity — weights pushed to extremes $\{-c, +c\}$
- ▶ Causes exploding and vanishing gradients
- ▶ Sensitive to choice of clipping constant c

- **Key idea:** enforce Lipschitz constraint via **gradient penalty** instead of weight clipping

- ▶ Optimal critic has gradient norm = 1 at interpolated points $\hat{x} = \epsilon x + (1 - \epsilon)G(z), \epsilon \sim \mathcal{U}[0, 1]$
- ▶ Penalize deviation of gradient norm from 1 directly

WGAN-GP: GRADIENT PENALTY I

innovate

achieve

lead

- **WGAN-GP objective:**

$$\mathcal{L} = \underbrace{\mathbb{E}[f(G(z))] - \mathbb{E}[f(x)]}_{\text{WGAN loss}} + \lambda \underbrace{\mathbb{E}\left[\left(\|\nabla_{\hat{x}} f(\hat{x})\|_2 - 1\right)^2\right]}_{\text{gradient penalty}}$$

- **Achieves:** better training stability than weight clipping, no hyperparameter sensitivity, high quality samples on par with DCGAN

SPECTRAL NORMALIZATION GAN (SNGAN)

- **Problem:** gradient penalty requires extra backward pass per iteration — computationally expensive
- **Key idea:** enforce Lipschitz constraint by **normalizing weight matrices** using their spectral norm
 - ▶ Spectral norm of a matrix $W \equiv$ largest singular value $\sigma(W)$
 - ▶ Normalize each layer: $\hat{W} = W/\sigma(W)$
 - ▶ Efficiently approximated via **power iteration** — single forward pass
- **Applied to:** discriminator weight matrices (later SAGAN extends to generator)
- **Achieves:**
 - ▶ Lipschitz constraint without extra backward pass
 - ▶ Computational benefit over WGAN-GP
 - ▶ First GAN to work on full ImageNet (1M images)
 - ▶ High quality class-conditional samples at ImageNet scale

DEEP CONVOLUTIONAL GAN (DCGAN)

- **Problem:** vanilla GAN uses fully connected layers — no spatial structure, unstable on images
- **Key idea:** replace FC layers with **convolutional architecture** designed for stable training
 - ▶ **Generator:** transposed convolutions to upsample $z \in \mathbb{R}^{100} \rightarrow 64 \times 64$ image
 - ▶ **Discriminator:** strided convolutions to downsample
- **Architecture guidelines** (Radford et al., 2016):
 - ▶ No max-pooling — replace with strided/transposed convolutions
 - ▶ BatchNorm in both G and D — except output of G and input of D

DEEP CONVOLUTIONAL GAN (DCGAN) II

- ▶ ReLU in G (Tanh at output); LeakyReLU (0.2) in D (Sigmoid at output)
- ▶ Adam: $\text{lr} = 2 \times 10^{-4}$, momentum = 0.5, batch size = 128
- **Achieves**: first stable GAN on large image datasets, smooth latent space, representation learning

DCGAN: RESULTS & LATENT SPACE STRUCTURE

- **Sample quality**: good samples on 3M image datasets (faces, bedrooms) for the first time
- **Smooth interpolations**: walking in z -space produces semantically smooth transitions — no sharp jumps
 - ▶ Confirms G has learned a meaningful latent representation
 - ▶ Not memorizing training data
- **Vector arithmetic in z -space**:
 - ▶ $z_{\text{smiling woman}} - z_{\text{neutral woman}} + z_{\text{neutral man}} \approx z_{\text{smiling man}}$
 - ▶ $z_{\text{man with glasses}} - z_{\text{man}} + z_{\text{woman}} \approx z_{\text{woman with glasses}}$
 - ▶ Semantic concepts encoded as **linear directions** in z -space
- **Remaining problems**:
 - ▶ Unstable training — brittle to hyperparameter choices
 - ▶ No control over generated attributes
 - ▶ Limited resolution

SELF-ATTENTION GAN (SAGAN)

innovate

achieve

lead

- **Problem:** convolutional layers have **limited receptive field** — cannot model long-range spatial dependencies
 - ▶ e.g. fur texture on a dog's back must be consistent with its face
 - ▶ CNNs struggle — local filters miss global structure
- **Key ideas:**
 - ▶ Self-attention in both G and D — each spatial location attends to all others
 - ▶ Spectral normalization applied to **both** G and D weight matrices (previously only D)
 - ▶ Hinge loss for more stable adversarial training
- **Achieves:**

SELF-ATTENTION GAN (SAGAN)



- ▶ First GAN to produce good **unconditional** full ImageNet samples
- ▶ Geometrically consistent global structure in generated images
- ▶ Foundation for BigGAN

BIGGAN: LARGE SCALE GAN TRAINING

- **Problem:** SAGAN produces good samples but limited resolution and fidelity at scale
- **Key ideas** (Brock et al., 2019):
 - ▶ Scale up SAGAN — $2\times$ parameters, $8\times$ batch size
 - ▶ Shared class embeddings — class vector fed to all layers of G
 - ▶ Noise vector skip connections — z injected at multiple resolutions
 - ▶ Orthogonal regularization — keeps G weights near orthogonal matrix, improves stability
 - ▶ Truncation trick — sample z from truncated normal at inference
 - ★ Small truncation: high fidelity, low diversity
 - ★ Large truncation: high diversity, lower fidelity

BIGGAN: LARGE SCALE GAN TRAINING II

- **Achieves:** 128×128 , 256×256 , 512×512 high-fidelity ImageNet images — state-of-the-art FID and IS at time of publication

CONDITIONAL GAN (cGAN)



- **Problem:** vanilla GAN has no control over what is generated — z determines output randomly
- **Key idea:** condition both G and D on auxiliary information y (class label, text, image)
 - ▶ G : takes both noise z and label $y \Rightarrow$ generates $\hat{x}$ of class y
 - ▶ D : takes both sample and label $y \Rightarrow$ classifies real/fake *given* y
- **Objective:**

$$\min_G \max_D \mathbb{E}[\log D(x, y)] + \mathbb{E}[\log(1 - D(G(z, y), y))]$$

- **ACGAN variant:** discriminator also predicts class label — adds classification loss to both G and D
- **Achieves:** controlled generation — generate specific classes, styles, or attributes on demand

CYCLEGAN: UNPAIRED IMAGE TRANSLATION

- **Problem:** paired image translation (pix2pix) requires matched image pairs — expensive and often unavailable
- **Key idea:** learn translation between two domains X and Y using **unpaired data** via cycle consistency
 - ▶ Two generators: $G : X \rightarrow Y$ and $F : Y \rightarrow X$
 - ▶ Two discriminators: D_X and D_Y
 - ▶ **Cycle consistency:** $F(G(x)) \approx x$ and $G(F(y)) \approx y$

- **Full objective:**

$$\mathcal{L} = \mathcal{L}_{\text{adv}}(G, D_Y) + \mathcal{L}_{\text{adv}}(F, D_X) + \lambda \mathcal{L}_{\text{cyc}}(G, F)$$

- **Applications:** horse $\leftrightarrow$ zebra, summer $\leftrightarrow$ winter, day $\leftrightarrow$ night, photo $\leftrightarrow$ painting
- **Achieves:** unpaired domain transfer — **AV application:** synthetic day $\rightarrow$ night camera adaptation without paired sensor data

STYLEGAN: STYLE-BASED GENERATOR

- **Problem:** in DCGAN/ProGAN, z controls all aspects of generation entangled — cannot independently control coarse (pose) vs fine (texture) attributes
- **Key ideas** (Karras et al., 2019):
 - ▶ **Mapping network:** $z \rightarrow w$ via 8-layer MLP — $\mathcal{W}$ -space is more disentangled than $\mathcal{Z}$ -space
 - ▶ **AdaIN (Adaptive Instance Normalization):** style vector w modulates each layer's feature maps

$$\text{AdaIN}(x_i, y) = y_{s,i} \frac{x_i - \mu(x_i)}{\sigma(x_i)} + y_{b,i}$$

- ▶ **Noise injection** at each layer — controls stochastic detail
 - ▶ Different layers control different scales: coarse (pose) $\rightarrow$ middle (features) $\rightarrow$ fine (texture)
- **Achieves:** disentangled style control, style mixing, high fidelity face synthesis

- **StyleGAN problem:** water droplet-like artifacts in generated images — caused by instance normalization creating strong localized features
- **StyleGAN2** (Karras et al., 2020):
 - ▶ Replace AdaIN with **weight demodulation** — normalizes conv weights instead of feature maps
 - ▶ Removes blob artifacts
 - ▶ Lazy regularization — compute path length penalty less frequently
 - ▶ **Achieves:** cleaner high-resolution synthesis, improved FID on FFHQ

STYLEGAN3



- **StyleGAN2 problem:** texture sticking — fine details fixed to screen coordinates, not object surface
- **StyleGAN3** (Karras et al., 2021):
 - ▶ Fully **equivariant** architecture — translation and rotation equivariant at subpixel scale
 - ▶ Internal activations encode phase, not position
 - ▶ Requires custom CUDA kernel
 - ▶ **Achieves:** texture moves naturally with object, alias-free generation

TABLE OF CONTENTS



1 Motivation & Big Picture

2 GAN Foundations

3 GAN Variants

4 Case Study: AV Prototype

5 Wrap-Up



CASE STUDY: SYNTHETIC PERCEPTION DATA

- **Problem:** AV perception models require diverse training data — rare scenarios (fog, night, rain) are hard to collect
- **GAN solution:**
 - ▶ **CycleGAN:** unpaired day → night, clear → foggy camera adaptation — no paired sensor data needed
 - ▶ **DriveGAN:** controllable neural driving simulator — generates photorealistic driving scenes conditioned on control inputs
 - ▶ **GAIA-1:** world model combining video, text, action — generates diverse AV scenarios for training
- **Pipeline:**
 - ▶ Real daytime LiDAR + camera data → CycleGAN → synthetic night/fog data
 - ▶ Augment training set ⇒ improved perception in rare conditions
- **Outcome:** reduced data collection cost, improved model robustness across weather and lighting conditions

CASE STUDY: RARE FAULT SYNTHESIS

- **Problem:** rare fault events (wheel slip, brake failure, engine overheating) are underrepresented in telemetry logs — predictive maintenance models fail on rare classes
- **GAN solution:**
 - ▶ **cGAN:** condition on fault type $y \Rightarrow$ generate synthetic telemetry sequences of specific rare fault classes
 - ▶ **WGAN-GP:** stable training on time-series data — avoids mode collapse on imbalanced fault distribution
- **Pipeline:**
 - ▶ Train cGAN on available fault logs
 - ▶ Generate synthetic rare fault sequences
 - ▶ Augment training set for downstream classifier
- **Outcome:** improved fault detection recall on rare classes, reduced dependency on real fault data collection

CASE STUDY: SYNTHETIC MARKET DATA

- **Problem:** automotive sales forecasting requires diverse market scenarios — historical data is limited and region-specific
- **GAN solution:**
 - ▶ **TimeGAN:** generates synthetic time-series of dealer sales, inventory, and demand — preserves temporal dynamics
 - ▶ **cGAN:** condition on region, season, vehicle segment $\Rightarrow$ targeted scenario generation
- **Pipeline:**
 - ▶ Historical sales data $\rightarrow$ TimeGAN $\rightarrow$ synthetic market scenarios
 - ▶ Feed synthetic data into demand forecasting model
 - ▶ Evaluate on held-out real regional data
- **Outcome:** richer training distribution for forecasting models, scenario planning for low-data regions

TABLE OF CONTENTS



1 Motivation & Big Picture

2 GAN Foundations

3 GAN Variants

4 Case Study: AV Prototype

5 Wrap-Up

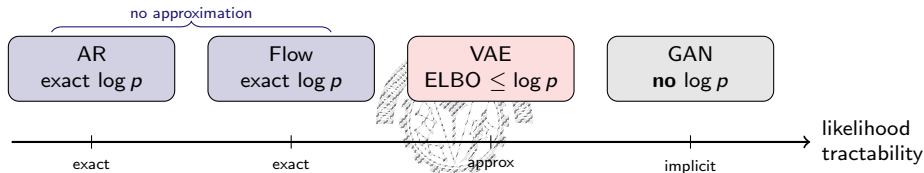


SUMMARY: GAN LANDSCAPE



Model	Problem solved	Key idea	Loss	Quality
GAN	Implicit generation	Minimax game	JS div.	Baseline
WGAN	Vanishing gradient	EM distance	Wasserstein	Stable
WGAN-GP	Weight clipping	Gradient penalty	$W + GP$	Better
SNGAN	Compute cost	Spectral norm	Hinge	ImageNet
DCGAN	FC on images	Conv architecture	BCE	Good
SAGAN	Local receptive field	Self-attention	Hinge	Full IN
BigGAN	Scale + fidelity	Truncation trick	Hinge	512px
cGAN	No control	Label conditioning	BCE	Controlled
CycleGAN	Paired data	Cycle consistency	Adv+Cyc	Unpaired
StyleGAN	Entangled style	AdaIN + mapping	—	SOTA

ELBO THREAD: GAN IN THE GENERATIVE MODEL LANDSCAPE



	AR	Flow	VAE	GAN
Likelihood	Exact	Exact	ELBO	None (implicit)
Sampling speed	Slow	Fast	Fast	Fast
Sample quality	Good	Good	Blurry	Sharp
Training stability	Stable	Stable	Stable	Unstable
Latent space	None	Yes	Yes	z only

REFERENCES



- Goodfellow et al., *Generative Adversarial Networks*, NeurIPS 2014
- Radford et al., *Unsupervised Representation Learning with DCGAN*, ICLR 2016
- Arjovsky et al., *Wasserstein GAN*, ICML 2017
- Gulrajani et al., *Improved Training of WGANs*, NeurIPS 2017
- Miyato et al., *Spectral Normalization for GANs*, ICLR 2018
- Zhang et al., *Self-Attention GAN*, ICML 2019
- Brock et al., *Large Scale GAN Training*, ICLR 2019
- Mirza & Osindero, *Conditional GAN*, arXiv 2014
- Zhu et al., *Unpaired Image-to-Image Translation using CycleGAN*, ICCV 2017
- Karras et al., *A Style-Based Generator Architecture*, CVPR 2019
- Karras et al., *Analyzing and Improving StyleGAN*, CVPR 2020
- Karras et al., *Alias-Free GAN (StyleGAN3)*, NeurIPS 2021
- Goodfellow et al., *Deep Learning*, MIT Press 2016



Thank you :)